

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Operating Systems

COURSE CODE: CSA0401

COURSE OUTCOME COVERED

CO4: Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.

Assessment Tool Weightage: 30%

QUESTIONS: 5

Total Marks:25

Annexure A

COMPARATIVE ANALYSIS

Code of Conduct:

I G. Yashmita (192511032) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G. Yashmita

Annexure A

COMPARATIVE ANALYSIS

S.No	Comparative Analysis Questions	Marks
1	Compare Contiguous and Linked file allocation methods in terms of access speed, memory utilization, and fragmentation. Identify which method is more suitable for sequential file access and justify your answer.	5
2	Differentiate between Linked and Indexed file allocation with respect to random access capability, storage overhead, and reliability. Explain which method is more efficient for large files.	5

3	Compare FCFS and SSTF disk scheduling algorithms based on seek time, fairness, and the possibility of starvation. Discuss which algorithm performs better under heavy disk load.	5
4	Analyze the differences between SCAN and C-SCAN disk scheduling algorithms in terms of head movement, response time, and uniformity of service. Which algorithm is more suitable for real-time systems and why?	5
5	Compare LOOK and C-LOOK disk scheduling algorithms with respect to efficiency, total head movement, and practical implementation. Explain which algorithm provides better performance and under what conditions.	5

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Contiguous vs. Linked File Allocation

Contiguous and linked allocation are two methods used by an operating system to allocate disk blocks to files. In contiguous allocation, all blocks of a file are stored next to each other. In linked allocation, the blocks can be stored anywhere on the disk and are connected using pointers.

Comparison Point	Contiguous Allocation	Linked Allocation
1. Access Speed	Provides very fast access because all blocks are stored consecutively. The disk head can read consecutive blocks with fewer movements.	Access is slower because the system has to follow the pointer from one block to the next block.
2. Memory Utilization	Requires a continuous group of free blocks. Therefore, available space may not always be utilized effectively.	Blocks can be allocated anywhere on the disk, so free space can be utilized more efficiently.
3. Fragmentation	It suffers from external fragmentation , where free space is divided into small unused areas.	It does not suffer from external fragmentation because blocks do not need to be consecutive.
4. File Growth	File expansion is difficult if there are no free blocks immediately after the existing file. The file may need to be moved.	File growth is easy because additional blocks can be allocated anywhere and linked to the existing blocks.
5. Sequential Access	Excellent for sequential access because blocks are consecutive, resulting in fewer seek operations and faster data transfer.	Suitable for sequential access, but pointer traversal introduces additional overhead.

Conclusion:

Contiguous allocation is more suitable for sequential file access because the blocks are stored continuously. This reduces disk-head movement, seek time, and access overhead. However, linked allocation is more flexible for files that frequently change in size.

2. Linked vs. Indexed File Allocation

Linked and indexed allocation are used to organize file blocks on a disk. Linked allocation connects individual blocks using pointers, whereas indexed allocation uses a separate index block that stores the addresses of the file's data blocks.

Comparison Point	Linked Allocation	Indexed Allocation
1. Random Access Capability	Random access is inefficient because the system must start from the first block and follow pointers until it reaches the required block.	Random access is efficient because the index block directly contains the addresses of the required data blocks.
2. Storage Overhead	Each data block needs space to store a pointer to the next block, reducing the amount of space available for actual data.	Requires additional space for index blocks to store block addresses.
3. Reliability	If a pointer is corrupted or lost, the remaining linked blocks may become inaccessible.	Provides better access reliability because block addresses are maintained in the index structure.
4. Large File Support	Large files can be stored, but accessing blocks becomes slower because many pointers may have to be followed.	Well suited for large files. Multi-level indexing can be used when a single index block cannot hold all addresses.
5. Access Efficiency	More efficient for sequential access because blocks can be followed one after another.	More efficient for both random and direct access because the required block can be located through the index.

Conclusion:

Indexed allocation is more efficient for large files, especially when random access is frequently required. Although it requires additional storage for index blocks, it provides faster access and better organization of file blocks than linked allocation.

3. FCFS vs. SSTF Disk Scheduling

FCFS (First-Come, First-Served) and SSTF (Shortest Seek Time First) are disk scheduling algorithms used to determine the order in which disk requests are serviced. FCFS follows the arrival order, while SSTF selects the request nearest to the current disk-head position.

Comparison Point	FCFS	SSTF
1. Working Principle	Serves disk requests in the exact order in which they arrive, without considering their location on the disk.	Selects and services the request that requires the shortest seek distance from the current head position.
2. Seek Time	Usually has a higher average seek time because the next request may be located far from the current head position.	Usually provides lower average seek time because the closest request is selected first.

Comparison Point	FCFS	SSTF
3. Fairness	Highly fair because every request is handled according to its arrival order.	Less fair because requests close to the head may continuously receive priority over distant requests.
4. Starvation	Starvation does not occur because every request is eventually serviced in order.	Starvation is possible if new requests continuously arrive near the current head position.
5. Heavy Disk Load	Under heavy load, it may produce large head movements and increased waiting time.	Generally performs better under heavy load because selecting nearby requests reduces average head movement and seek time.

Conclusion:

SSTF generally performs better than FCFS under heavy disk load because it reduces average seek time and improves disk performance. However, FCFS is more suitable when fairness is important because it does not cause starvation. SSTF provides better performance at the cost of fairness.

4. SCAN vs. C-SCAN Disk Scheduling

SCAN and C-SCAN are disk scheduling algorithms designed to reduce excessive head movement and provide better performance than simple FCFS. SCAN moves the disk head in both directions, whereas C-SCAN services requests in only one direction.

Comparison Point	SCAN	C-SCAN
1. Head Movement	The disk head moves in one direction, services requests, reaches the last request, and then reverses direction.	The disk head services requests in one direction. After reaching the last request, it returns to the beginning and starts servicing again in the same direction.
2. Response Time	Response time depends on the position of a request and the current direction of the head.	Provides a more predictable and uniform response time because requests are serviced in a circular order.
3. Uniformity of Service	Service is reasonably fair, but requests in the middle region may receive service more frequently.	Provides more uniform service because every request waits for approximately one complete cycle.
4. Head Movement Efficiency	Can be efficient because requests are serviced while the head moves in both directions.	May involve additional movement when the head returns to the beginning without servicing requests.

Comparison Point	SCAN	C-SCAN
5. Suitable Application	Suitable for general-purpose systems where good performance and reasonable fairness are required.	Suitable for systems where predictable and uniform response time is more important.

Conclusion:

C-SCAN is more suitable for real-time or time-sensitive systems when uniform response time is required. It provides more predictable waiting times than SCAN. SCAN can be more efficient in some situations because it services requests during both forward and reverse movements.

5. LOOK vs. C-LOOK Disk Scheduling

LOOK and C-LOOK are improved versions of SCAN and C-SCAN. They avoid unnecessary movement of the disk head to the physical end of the disk when there are no pending requests in that direction.

Comparison Point	LOOK	C-LOOK
1. Working Principle	The head moves in one direction and services requests until the last pending request, then reverses direction.	The head moves in one direction and services requests until the last pending request, then jumps back to the first pending request.
2. Efficiency	Efficient because the head does not travel to the physical end of the disk when there are no requests there.	Also highly efficient because it avoids unnecessary movement to the disk ends and follows a circular servicing pattern.
3. Total Head Movement	Generally requires less movement than SCAN because it stops at the last pending request before reversing.	Generally requires less movement than C-SCAN because it does not travel to the physical disk boundaries unnecessarily.
4. Response Time	Response time can vary because requests are serviced in both directions.	Provides more uniform response time because requests are serviced in a single direction.
5. Practical Implementation	Simpler because the head reverses direction after reaching the last pending request.	Slightly more complex because the head must jump from the last request to the first request.

Conclusion:

LOOK provides better performance when minimizing total head movement is the main objective, especially when requests are distributed across the disk. **C-LOOK** is preferable when uniform response time and predictable service are more important. Both algorithms improve upon SCAN and C-SCAN by avoiding unnecessary movement to the physical ends of the disk.